

รายงานผลการทดลอง

Lab 9: Spring Boot - Transaction

ระบบฝากเงินอย่างง่าย (Simple Deposit System)

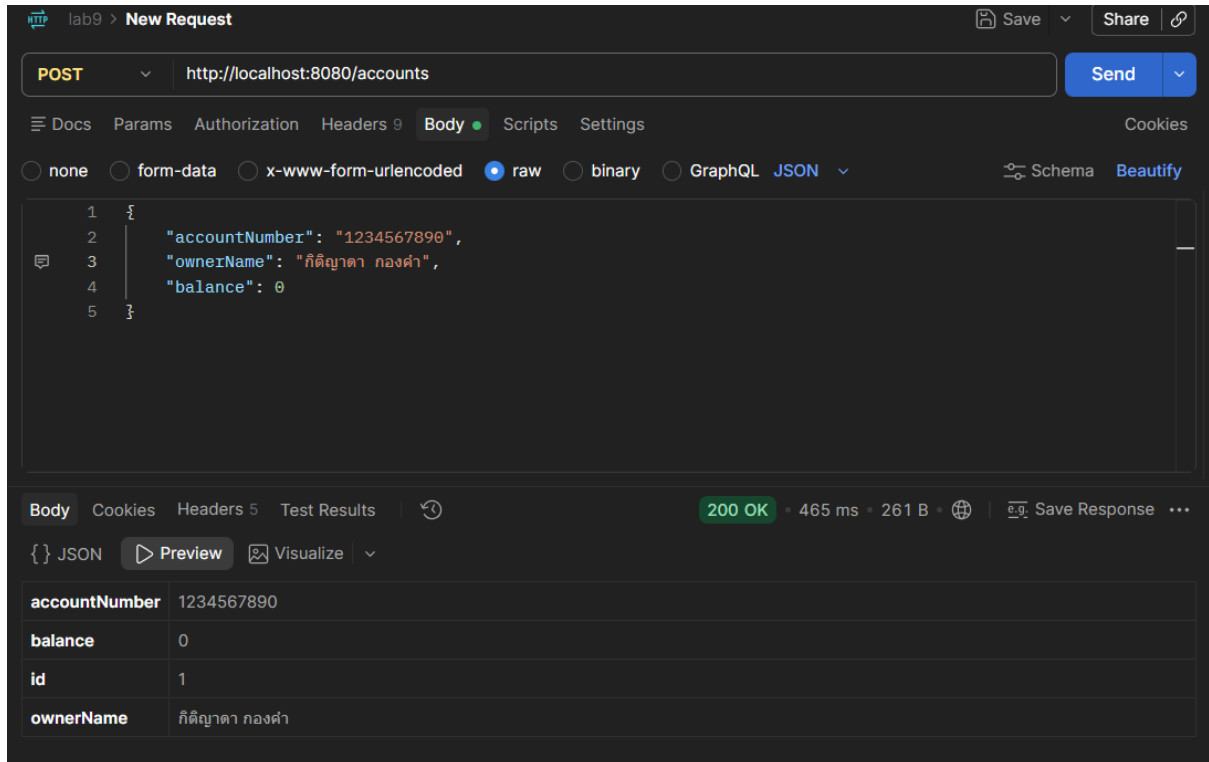
ชื่อ-นามสกุล: นางสาวกิตติมาดา กองคำ

รหัสนักศึกษา: 673380509-6

รายวิชา: Principle of Software Design

1. การสร้างบัญชี (Create Account)

ทดสอบ Endpoint `POST /accounts` เพื่อสร้างบัญชีธนาคารใหม่ โดยส่งข้อมูล `accountNumber`, `ownerName` และ `balance` เริ่มต้นผ่าน Postman

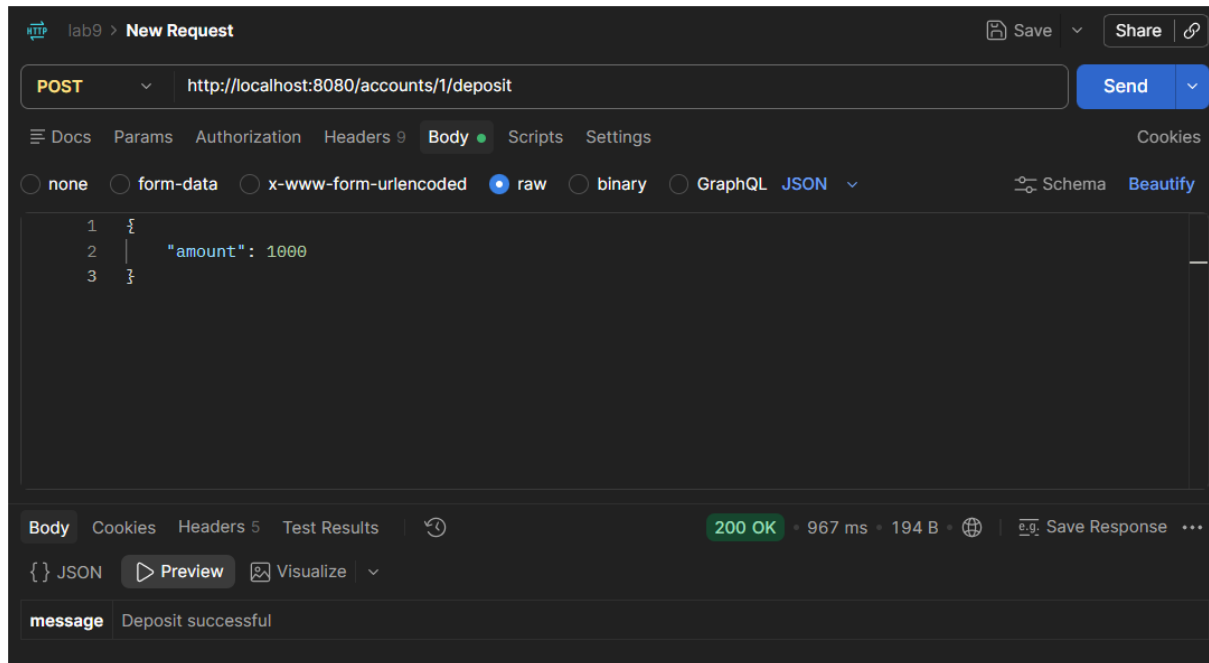


อธิบายผลการทดลอง:

- เมื่อส่ง Request `POST /accounts` พร้อมข้อมูลบัญชีในรูปแบบ JSON ระบบตอบกลับด้วย Status 200 OK
- Response ที่ได้แสดงข้อมูลบัญชีครบถ้วน ได้แก่ `accountNumber`, `ownerName`, `balance` และ `id` ที่ระบบสร้างให้อัตโนมัติ (Auto Increment)
- แสดงว่า `AccountController` และ `AccountService` ทำงานถูกต้อง สามารถบันทึกข้อมูลบัญชีใหม่ลงตาราง `account` ในฐานข้อมูล PostgreSQL ได้สำเร็จ

2. การฝากเงินสำเร็จ (Successful Deposit)

ทดสอบ Endpoint `POST /accounts/{id}/deposit` เพื่อฝากเงินเข้าบัญชีที่สร้างไว้ในข้อ 1 โดยส่งจำนวนเงินที่ต้องการฝากผ่าน Postman



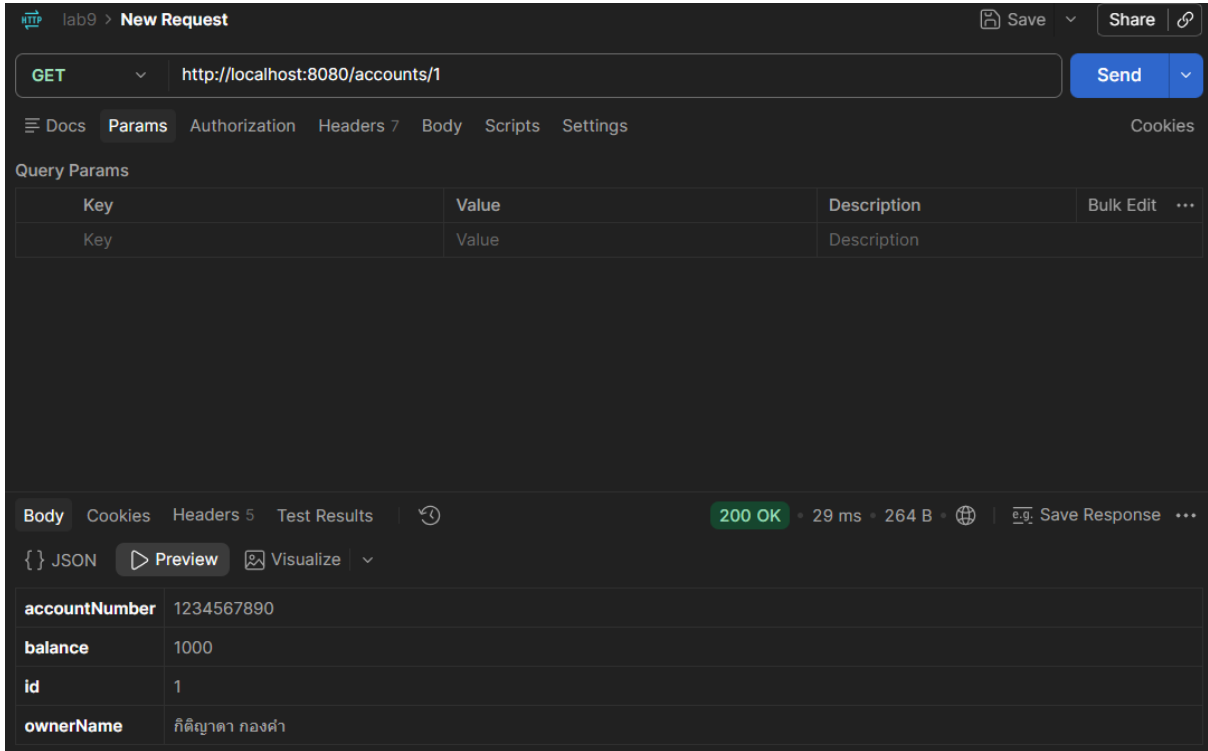
อธิบายผลการทดลอง:

- เมื่อส่ง Request `POST /accounts/1/deposit` พร้อมจำนวนเงินที่ต้องการฝาก ระบบตอบกลับด้วย Status 200 OK และข้อความ "Deposit successful"
- ภายในเมธอด `deposit()` ของ `DepositService` มีการทำงาน 2 ขั้นตอนพร้อมกันภายใต้ Transaction เดียวกัน คือ เพิ่มยอดเงินใน balance ของ Account และสร้างระเบียบ `DepositTransaction` ใหม่เพื่อบันทึกประวัติการฝาก
- การที่ Response กลับมาเป็น 200 OK แสดงว่าทั้งสองขั้นตอนทำงานสำเร็จและถูก COMMIT ลงฐานข้อมูลเรียบร้อยแล้ว

3. ตรวจสอบ Account หลังฝากเงิน

ทดสอบ Endpoint `GET /accounts/{id}` เพื่อตรวจสอบว่ายอดเงิน (balance)

ในบัญชีเปลี่ยนแปลงตามจำนวนเงินที่ฝากไป/จริงหรือไม่



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/accounts/1
- Status:** 200 OK
- Response Time:** 29 ms
- Response Size:** 264 B
- Response Type:** JSON
- Response Body:**

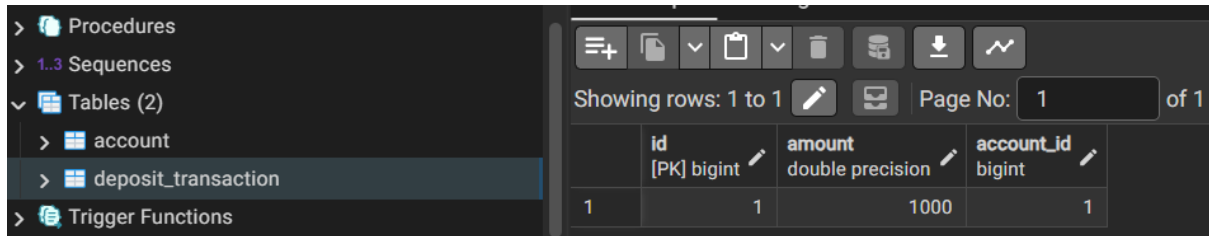
```
{  "accountNumber": 1234567890,  "balance": 1000,  "id": 1,  "ownerName": "กิตติญาดา กองคำ"}
```

อธิบายผลการทดลอง:

- เมื่อเรียก `GET /accounts/1` ระบบตอบกลับข้อมูลบัญชีพร้อมค่า balance ที่เพิ่มขึ้นจากเดิม (เช่น จาก 0 เป็น 1000) ตรงตามจำนวนเงินที่ฝากในข้อ 2
- ผลลัพธ์ยืนยันว่าขั้นตอน "เพิ่มยอดเงินในบัญชี" ภายในเมธอด `deposit()` ทำงานถูกต้องและข้อมูลถูกบันทึกลงฐานข้อมูลอย่างถาวร (Persist)

4. ตรวจสอบประวัติการฝากเงิน (DepositTransaction)

ตรวจสอบตาราง `deposit_transaction` ผ่าน pgAdmin ด้วยคำสั่ง `SELECT * FROM deposit_transaction;` เพื่อยืนยันว่าการบันทึกประวัติการฝากเงินจริง



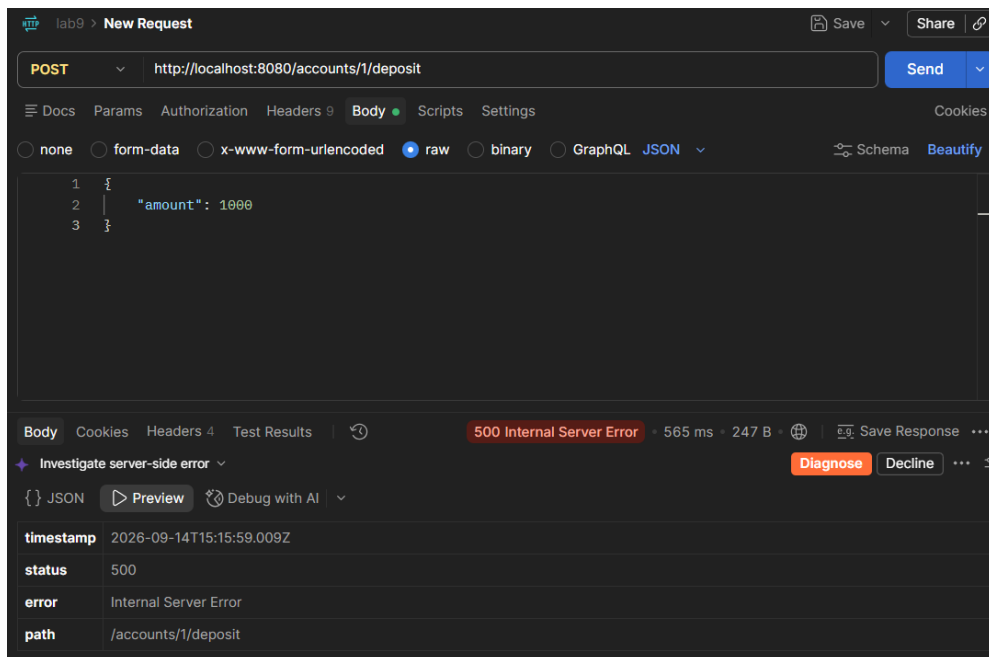
	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

อธิบายผลการทดลอง:

- ผลลัพธ์จากคำสั่ง `SELECT` พบข้อมูล 1 แถว ประกอบด้วย `id`, `amount` และ `account_id` ที่อ้างอิงกลับไปยังบัญชีที่ทำการฝากเงิน (`account_id = 1`)
- แสดงให้เห็นว่าขั้นตอน "บันทึกประวัติการฝากเงิน" ในเมธอด `deposit()` ทำงานสำเร็จ และความสัมพันธ์แบบ Many-to-One ระหว่าง `DepositTransaction` กับ `Account` ผ่านคอลัมน์ `account_id` ทำงานถูกต้องตามที่ออกแบบไว้

5. ทดลอง Transaction Rollback

จำลองสถานการณ์เกิดข้อผิดพลาดหลังบันทึกข้อมูลไปแล้วบางส่วน โดยเพิ่มคำสั่ง `throw new RuntimeException("Test Rollback");` ไว้ที่บรรทัดสุดท้ายของเมธอด `deposit()` (ขณะที่ยังมี `@Transactional` อยู่) แล้วยิง Request ฝากเงินซ้ำ



lab9 > New Request

POST http://localhost:8080/accounts/1/deposit

Body: { "amount": 1000 }

500 Internal Server Error · 565 ms · 247 B

Investigate server-side error

timestamp: 2026-09-14T15:59:00Z

status: 500

error: Internal Server Error

path: /accounts/1/deposit

The top screenshot shows a REST client response with status 200 OK. The response body is a JSON object:

```
{
  "accountNumber": 1234567890,
  "balance": 1000,
  "id": 1,
  "ownerName": "กิติญาดา กองคำ"
}
```

The bottom screenshot shows the pgAdmin interface. The left sidebar shows the database structure with 'deposit_transaction' selected. The right pane shows a table with the following data:

id	amount	account_id
1	1000	1

อธิบายผลการทดลอง:

- หลังเพิ่ม throw new RuntimeException() ต่อจากขั้นตอนบันทึกข้อมูลทั้งสองอย่างแล้วยิง Request เข้าระบบตอบกลับด้วย Status 500 Internal Server Error
- เมื่อตรวจสอบ balance ของบัญชีด้วย GET /accounts/1 พบว่ายอดเงินยังคงเท่าเดิม ไม่เพิ่มขึ้นตามจำนวนที่พยายามฝากในครั้งนี้
- เมื่อตรวจสอบตาราง deposit_transaction ใน pgAdmin พบว่ายังคงมีจำนวนแถวเท่าเดิม ไม่มีรายการใหม่ถูกเพิ่มเข้ามา
- ผลการทดลองนี้ยืนยันว่า @Transactional ทำงานถูกต้อง กล่าวคือเมื่อเกิดข้อผิดพลาด (Exception) ขึ้นภายในเมธอดที่มี @Transactional ระบบจะสั่ง ROLLBACK ย้อนการเปลี่ยนแปลงข้อมูลทั้งหมดในเมธอดนั้นกลับสู่สถานะก่อนเริ่ม Transaction แม้ว่าคำสั่ง save() จะถูกเรียกไปแล้วทั้งสองครั้งก่อนเกิด Error ก็ตาม

6. เปรียบเทียบ กรณีมี @Transactional และไม่มี @Transactional

ทดลองลบ @Transactional ออกจากเมธอด deposit() โดยยังคงบรรทัด throw new RuntimeException("Test Rollback"); ไว้เหมือนเดิม แล้วทดลองฝากเงินอีกครั้ง

The screenshot shows two Postman requests to the endpoint `http://localhost:8080/accounts/1/deposit`.

Top Request (Failed):

- Method: POST
- URL: `http://localhost:8080/accounts/1/deposit`
- Body (raw):

```
{
  "amount": 1000
}
```
- Status: 500 Internal Server Error
- Response (JSON):

```
{
  "timestamp": "2026-09-14T15:13:09.500Z",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/accounts/1/deposit"
}
```

Bottom Request (Successful):

- Method: POST
- URL: `http://localhost:8080/accounts/1/deposit`
- Status: 200 OK
- Response (JSON):

```
{
  "accountNumber": 1234567890,
  "balance": 2000,
  "id": 1,
  "ownerName": "กิติญาดา กองคำ"
}
```

อธิบายผลการทดลอง:

- กรณีมี @Transactional: เมื่อเกิด Error หลังบันทึกข้อมูล ระบบ ROLLBACK ข้อมูลทั้งหมดกลับคืน balance และตาราง deposit_transaction ไม่มีการเปลี่ยนแปลงใด ๆ
- กรณีไม่มี @Transactional: เมื่อเกิด Error ในลักษณะเดียวกัน คำสั่ง save() แต่ละคำสั่งจะถูกบันทึกลงฐานข้อมูลทันทีที่คำสั่งโดยอิสระต่อกัน ดังนั้นแม้จะเกิด Error ในภายหลัง ข้อมูลที่บันทึกไปแล้วก่อนหน้า (เช่น balance ที่เพิ่มขึ้น และ/หรือ DepositTransaction ที่ถูกสร้าง) จะยังคงอยู่ในฐานข้อมูล ไม่ถูกย้อนกลับ

- สรุปได้ว่า @Transactional ทำหน้าที่รวมหลายคำสั่งที่เกี่ยวข้องกันให้เป็นหน่วยงานเดียวกัน (Atomic Unit) หากขาด Annotation นี้ ข้อมูลในระบบมีความเสี่ยงที่จะไม่สอดคล้องกัน (Data Inconsistency) เมื่อเกิดข้อผิดพลาดระหว่างการทำงาน

7. ตอบคำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

@Transactional เป็น Annotation ที่ใช้กำหนดขอบเขตของ Transaction ให้กับเมธอด

โดยทำให้ทุกคำสั่งที่เกี่ยวข้องกับฐานข้อมูลภายในเมธอดนั้น ถูกนับรวมเป็นหน่วยงานเดียวกัน (Atomic Unit)

หากทุกคำสั่งทำงานสำเร็จ ระบบจะส่ง COMMIT เพื่อบันทึกข้อมูลทั้งหมดอย่างถาวร แต่หากมีคำสั่งใดคำสั่งหนึ่งเกิดข้อผิดพลาด ระบบจะส่ง ROLLBACK เพื่อย้อนการเปลี่ยนแปลงข้อมูลทั้งหมดกลับสู่สถานะก่อนเริ่ม Transaction

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน 1 ครั้งต้องเปลี่ยนแปลงข้อมูลมากกว่า 1 อย่างพร้อมกัน คือ เพิ่มยอดเงินในบัญชี (Account)

และบันทึกประวัติการฝากเงิน (DepositTransaction) ทั้งสองขั้นตอนนี้ควรสำเร็จไปพร้อมกันเสมอ

หากขั้นตอนใดขั้นตอนหนึ่งล้มเหลวแต่อีกขั้นตอนหนึ่งบันทึกไปแล้ว จะทำให้ข้อมูลในระบบไม่สอดคล้องกัน (เช่น

มีประวัติการฝากแต่ยอดเงินไม่เพิ่ม หรือยอดเงินเพิ่มแต่ไม่มีประวัติ) การใช้ Transaction

จึงช่วยรับประกันว่าการเปลี่ยนแปลงข้อมูลทั้งหมดจะสำเร็จหรือถูกยกเลิกไปพร้อมกันเสมอ

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันบันทึกการเปลี่ยนแปลงข้อมูลทั้งหมดที่เกิดขึ้นภายใน Transaction ลงฐานข้อมูลอย่างถาวร

ซึ่งจะเกิดขึ้นเมื่อทุกคำสั่งใน Transaction ทำงานสำเร็จโดยไม่มีข้อผิดพลาด ส่วน ROLLBACK

คือการยกเลิกและย้อนการเปลี่ยนแปลงข้อมูลทั้งหมดที่เกิดขึ้นภายใน Transaction กลับสู่สถานะก่อนเริ่มต้น

ซึ่งจะเกิดขึ้นเมื่อมีข้อผิดพลาด (Exception) เกิดขึ้นระหว่างการทำงานของ Transaction นั้น

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

มีความสัมพันธ์แบบ One-to-Many กล่าวคือ 1 Account สามารถมี DepositTransaction ได้หลายรายการ โดยฝั่ง

DepositTransaction จะเป็นฝั่ง Many-to-One ที่มีฟิลด์อ้างอิงกลับไปยัง Account ผ่านคอลัมน์ Foreign Key ชื่อ account_id

ซึ่งเพียงพอสำหรับการกำหนดความสัมพันธ์ในระดับฐานข้อมูล จึงไม่จำเป็นต้องเก็บลิสต์ของ DepositTransaction ไว้ในฝั่ง

Account (เพื่อหลีกเลี่ยงปัญหาการอ่านข้อมูลไม่รู้จบเมื่อแปลงเป็น JSON)

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

เมื่อเกิด Error ขึ้นภายในเมธอดที่มี @Transactional ระบบจะส่ง ROLLBACK ย้อนการเปลี่ยนแปลงข้อมูลทั้งหมดที่เกิดขึ้นใน

Transaction นั้นกลับคืน แม้ว่าคำสั่ง save() บางคำสั่งจะถูกเรียกไปแล้วก่อนเกิด Error ก็ตาม

ผลลัพธ์คือข้อมูลในฐานข้อมูลจะไม่มีการเปลี่ยนแปลงใด ๆ เลย เสมือนว่าไม่เคยมีการฝากเงินครั้งนั้นเกิดขึ้น

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อลบ @Transactional ออกและเกิด Error ในลักษณะเดียวกัน คำสั่ง save()

แต่ละคำสั่งจะทำงานและบันทึกข้อมูลลงฐานข้อมูลจบไปที่ละคำสั่งทันที โดยไม่มีการมองภาพรวมว่างานทั้งก้อนสำเร็จหรือไม่

ดังนั้นแม้จะเกิด Error ขึ้นภายหลัง ข้อมูลที่บันทึกไปแล้วก่อนหน้านี้ (เช่น ยอดเงินที่เพิ่มขึ้น) จะยังคงอยู่ในฐานข้อมูลโดยไม่ถูกย้อนกลับ ต่างจากกรณีที่มี @Transactional ซึ่งจะ ROLLBACK ข้อมูลทั้งหมดกลับคืนเมื่อเกิดข้อผิดพลาด แสดงให้เห็นความสำคัญของ @Transactional ในการรักษาความถูกต้องสอดคล้องกันของข้อมูล

สรุปผลการทดลอง

จากการทดลองทั้งหมด สามารถสรุปได้ว่า Spring Boot ร่วมกับ Spring Data JPA สามารถจัดการ Transaction ผ่าน Annotation @Transactional ได้อย่างมีประสิทธิภาพ โดย Annotation นี้ควรเขียนไว้ที่ชั้น Service เนื่องจากเป็นชั้นเดียวที่ทราบขอบเขตของงานหนึ่งชิ้นตั้งแต่ต้นจนจบ การมี @Transactional ช่วยรับประกันความถูกต้องสอดคล้องกันของข้อมูล (Data Consistency) เมื่อมีการเปลี่ยนแปลงข้อมูลหลายส่วนพร้อมกัน หากเกิดข้อผิดพลาดระหว่างการทำงาน ระบบจะ ROLLBACK ข้อมูลทั้งหมดกลับคืนโดยอัตโนมัติ ในทางตรงกันข้าม หากไม่มี @Transactional การบันทึกข้อมูลแต่ละคำสั่งจะเป็นอิสระต่อกัน ซึ่งอาจนำไปสู่ปัญหาข้อมูลไม่สอดคล้องกันได้เมื่อเกิดข้อผิดพลาดกลางคัน